

hw4_local

March 20, 2025

Copying images to training, validation, and test directories

```
[ ]: %matplotlib inline
import pathlib
import tensorflow as tf
from tensorflow import keras
from keras import layers
from keras.applications import ResNet50
```

```
[ ]: new_base_dir = pathlib.Path("cats_vs_dogs_small")
```

Instantiating the ResNet50 convolutional base

```
[ ]: conv_base = ResNet50(
    weights="imagenet", # Pre-trained on ImageNet
    include_top=False, # Exclude the fully-connected layer at the top
    input_shape=(180, 180, 3) # Input shape for your images
)
```

```
[ ]: conv_base.summary()
```

Fast feature extraction without data augmentation Extracting the VGG16 features and corresponding labels

```
[ ]: import numpy as np

def get_features_and_labels(dataset):
    all_features = []
    all_labels = []
    for images, labels in dataset:
        preprocessed_images = keras.applications.resnet.preprocess_input(images)
        features = conv_base.predict(preprocessed_images)
        all_features.append(features)
        all_labels.append(labels)
    return np.concatenate(all_features), np.concatenate(all_labels)
```

```
[ ]: from keras.utils import image_dataset_from_directory

train_dataset = image_dataset_from_directory(
```

```

    new_base_dir / "train",
    image_size=(180, 180),
    batch_size=32)
validation_dataset = image_dataset_from_directory(
    new_base_dir / "validation",
    image_size=(180, 180),
    batch_size=32)
test_dataset = image_dataset_from_directory(
    new_base_dir / "test",
    image_size=(180, 180),
    batch_size=32)

```

```

[ ]: train_features, train_labels = get_features_and_labels(train_dataset)
    val_features, val_labels = get_features_and_labels(validation_dataset)
    test_features, test_labels = get_features_and_labels(test_dataset)

```

```

[ ]: random_numbers = np.random.normal(size=(1000, 16))
    dataset = tf.data.Dataset.from_tensor_slices(random_numbers)

```

```

[ ]: for i, element in enumerate(dataset):
    print(element.shape)
    if i >= 2:
        break

```

```

[ ]: batched_dataset = dataset.batch(32)
    for i, element in enumerate(batched_dataset):
        print(element.shape)
        if i >= 2:
            break

```

```

[ ]: reshaped_dataset = dataset.map(lambda x: tf.reshape(x, (4, 4)))
    for i, element in enumerate(reshaped_dataset):
        print(element.shape)
        if i >= 2:
            break

```

Displaying the shapes of the data and labels yielded by the Dataset

```

[ ]: for data_batch, labels_batch in train_dataset:
    print("data batch shape:", data_batch.shape)
    print("labels batch shape:", labels_batch.shape)
    break

```

Defining and training the densely connected classifier

```

[ ]: data_augmentation = keras.Sequential(
    [
        layers.RandomFlip("horizontal"),

```

```

        layers.RandomRotation(0.1),
        layers.RandomZoom(0.2),
    ]
)

inputs = keras.Input(shape=(180, 180, 3))
x = data_augmentation(inputs)
x = keras.applications.resnet.preprocess_input(x)
x = conv_base(x)
x = layers.Flatten()(x)
x = layers.Dense(256)(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)
conv_base.trainable = False

model.compile(loss="binary_crossentropy",
              optimizer="rmsprop",
              metrics=["accuracy"])

```

```

[ ]: callbacks = [
    keras.callbacks.ModelCheckpoint(
        filepath="resnet_from_scratch",
        save_best_only=True,
        monitor="val_loss")
]
history = model.fit(
    train_dataset,
    epochs=30,
    validation_data=validation_dataset,
    callbacks=callbacks)

```

```

[ ]: import matplotlib.pyplot as plt
acc = history.history["accuracy"]
val_acc = history.history["val_accuracy"]
loss = history.history["loss"]
val_loss = history.history["val_loss"]
epochs = range(1, len(acc) + 1)
plt.plot(epochs, acc, "bo", label="Training accuracy")
plt.plot(epochs, val_acc, "b", label="Validation accuracy")
plt.title("Training and validation accuracy")
plt.legend()
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss of ResNet50")
plt.legend()

```

```
plt.show()
```

```
[ ]: model = keras.models.load_model("resnet_from_scratch")
test_loss, test_acc = model.evaluate(test_dataset)
print(f"Test accuracy ResNet50: {test_acc:.3f}")
```

```
[ ]: conv_base.trainable = True
for layer in conv_base.layers[:-4]:
    layer.trainable = False
```

Fine-tuning the model

```
[ ]: model.compile(loss="binary_crossentropy",
                  optimizer=keras.optimizers.RMSprop(learning_rate=1e-5),
                  metrics=["accuracy"])

callbacks = [
    keras.callbacks.ModelCheckpoint(
        filepath="fine_tuning",
        save_best_only=True,
        monitor="val_loss")
]
history = model.fit(
    train_dataset,
    epochs=30,
    validation_data=validation_dataset,
    callbacks=callbacks)
```

```
[ ]: import matplotlib.pyplot as plt
acc = history.history["accuracy"]
val_acc = history.history["val_accuracy"]
loss = history.history["loss"]
val_loss = history.history["val_loss"]
epochs = range(1, len(acc) + 1)
plt.plot(epochs, acc, "bo", label="Training accuracy")
plt.plot(epochs, val_acc, "b", label="Validation accuracy")
plt.title("Training and validation accuracy")
plt.legend()
plt.figure()
plt.plot(epochs, loss, "bo", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss after Fine-Tuning")
plt.legend()
plt.show()
```

```
[ ]: model = keras.models.load_model("fine_tuning")
test_loss, test_acc = model.evaluate(test_dataset)
print(f"Test accuracy after fine-tuning: {test_acc:.3f}")
```